

Using Steganography to hide messages inside PDF files

SSN Project Report

Fahimeh Alizadeh - `Fahimeh.Alizadeh@os3.nl`
Nicolas Canceill - `Nicolas.Canceill@os3.nl`
Sebastian Dabkiewicz - `Sebastian.Dabkiewicz@os3.nl`
Diederik Vandevenne - `Diederik.Vandevenne@os3.nl`

December 30, 2012

Abstract

Steganography focuses on hiding information in such a way that the message is undetectable for outsiders and only appears to the sender and intended recipient.

Portable Document Format (PDF) steganography has not received as much attention as other techniques like image steganography because of the lower capacity and text-based file format, which make it harder to hide data. However some approaches have been made in the field of PDF steganography.

One of the current and most promising methods uses the TJ values, which are used to display text, in PDF files to hide data. The goal of the project was to improve the capacity and, if possible, the security of this method.

The TJ method is therefore carefully analysed for weaknesses. In the process of doing this, an implementation of this method was developed. Statistical analyses of the TJ values showed that the TJ method is not very strong and that hidden data can easily be detected. Based on the results of the many experiments that were performed, two different algorithms were composed. The first one has a lower capacity but is more secure. The second one offers a much higher embedding capacity while it still keeps the same level of security. Both algorithms are proposed as an alternative for the original TJ method.

Contents

1	Introduction	1
1.1	Research question	1
1.2	Related work	1
1.2.1	Hidden characters and objects	1
1.2.2	Hiding data in operator values	2
1.3	Main contributions of this paper	2
1.4	Outline	3
2	Portable Document Format	4
2.1	Compression	4
2.2	Operators	4
2.2.1	Tc operator	4
2.2.2	Tw operator	5
2.2.3	TJ operator	5
2.2.4	Comparison of operators	6
3	Implementation of the original method	7
3.1	Technical considerations	7
3.1.1	Python 2	7
3.1.2	Parsing the TJ operators	7
3.1.3	QPDF	7
3.1.4	User-friendliness	8
3.2	Detailing the original method	8
3.2.1	Generating a seed for the chaotic maps	8
3.2.2	Finding the end of the message	8
4	Evaluating the TJ method	9
4.1	Data set	9
4.2	Randomness of TJ values	10
4.3	The total line width	12
4.4	Usefulness of the Logistic Chaotic Maps	14
5	Patching and improving the TJ method	16
5.1	Comparison of different PDF writers	16
5.2	Data encryption	18
5.3	Number of used bits in TJ values	18
5.4	Using most of the TJ values	20
5.5	Compensating the line width by changing TJ values	21
5.6	Random start and input positions	22
5.7	The new algorithm	22
5.8	Evaluating the new algorithm	23
5.8.1	Randomness of TJ values for character pairs	23
5.8.2	Comparison of the available capacity	25
5.8.3	A capacity versus security trade-off	26
6	Conclusions	27
7	Further research	28

A List of Acronyms	29
References	29

List of Tables

1	Appearance of the Tc, Tw and TJ operators in different PDF files	6
---	--	---

List of Figures

1	Tc operator	4
2	Tw operator	5
3	TJ operator	5
4	Distribution of TJ space values in an one-column document	9
5	Distribution of TJ space values in a two-column document	10
6	Distribution of TJ space values in combination document	11
7	Distribution of TJ space values between [-16,16] in a Jaws PDF file	12
8	Distribution of TJ space values between [-16,16] in a Jaws PDF file containing hidden data	13
9	Character widths object	13
10	Line width frequency	14
11	Distribution of TJ space values in a PDFCreator PDF file	16
12	Distribution of TJ space values in a $\LaTeX$ PDF file	17
13	Distribution of TJ values in a $\LaTeX$ PDF stego_file with 4 bits input data without encryption	18
14	Distribution of TJ values in a $\LaTeX$ PDF stego_file with 4 bits encrypted input data	19
15	Distribution of TJ values in a $\LaTeX$ PDF stego_file with 3 bits input data without encryption	19
16	Distribution of TJ values in a $\LaTeX$ PDF stego_file with 3 bits encrypted input data	20
17	The output of a stego_file with 4 bits input data and with encryption	20
18	Percentage of TJ space values in a Jaws PDF file	21
19	Distribution of TJ values for the e-w pair in a $\LaTeX$ PDF file without hidden data	23
20	Distribution of TJ values for the e-w pair in a $\LaTeX$ PDF file with hidden data	24
21	Distribution of TJ values for the d-t pair in a $\LaTeX$ PDF file without hidden data	24
22	Distribution of TJ values for the d-t pair in a $\LaTeX$ PDF file with hidden data	25

1 Introduction

Steganography encompasses techniques for writing hidden messages. The intended purpose is that only the sender and receiver should be able to find the hidden message without attracting the attention of others. In addition, a secure steganographic method is able to hide the message in such a way that even when an object is suspected to contain a hidden message, the presence of this hidden data cannot be determined with a high certainty. Cryptography protects the confidentiality of information and communication. Steganography on the other hand protects the information and communication from being detected.

Most current steganographic methods use multimedia files like pictures, audio and video files to hide information. This is mostly because of the steganographic embedding capacity they provide. Capacity is together with security the most important property of a steganographic method.

Notwithstanding the popularity of multimedia files for steganographic purposes, other files, whether binary data files, executables or text based files, can also be used to hide information. The widespread use of PDF files can make its use for this purpose an interesting and practical solution. Although it may be harder to do this since there is usually less space available. The text based format of a PDF document can also be a limitation because it is easy to analyse its contents and it may be harder to actually hide data into it.

Several attempts have been made in the field of PDF steganography (see Section 1.2), but the presented solutions and implementations are not always very well described and / or published. Therefore it is hard to find out if the proposed method is performing in a good manner. More research in the field of PDF steganography is needed to verify or disprove the proposed method.

1.1 Research question

The goal of this project is to improve on the current steganographic methods in PDF files by adding more embedding capacity and, if possible, by creating a more secure method.

Therefore the following research question was formulated:

How can the steganographic embedding capacity in PDF files be increased by altering the existent algorithms while keeping the same level of security?

1.2 Related work

In order to get a clear view at the landscape of PDF steganography, we established a state of the art in this domain. An overview of the current techniques is presented in this section.

1.2.1 Hidden characters and objects

Some of the current techniques only focus on hiding data by using invisible PDF components. As a result, the data will be perfectly undetectable if the PDF is opened in a regular PDF viewer. These techniques are described in the paragraphs below.

between-word/between-character embedding I.-S. Lee and W.-H. Tsai present two algorithms in [1], making use of the non-breaking space with American Standard Code for Information Interchange (ASCII) code A0.

The first technique embeds data by changing a normal white space into an A0 space to encode 1, and leaves the regular white space to encode 0. It does not increase the file size at all, but the amount of data that can be embedded is very limited by the number of white spaces in the text.

The second technique takes advantage of the A0 character: by changing its width to zero, it appears totally invisible, so you can insert any amount between two characters without changing the appearance of the text. Data is embedded by inserting a number of zero-length spaces at each between-character location; the number of spaces encodes an ASCII character. This technique does increase the file size, but much more data can be embedded.

Incremental updates H. Liu *et al.* present three algorithms in [2], making use of the *incremental update* feature of PDF.

The first technique embeds data by altering text in a visible way (change the value of some text state variables), then writes an incremental update containing the original PDF data, so the altered text is not actually displayed.

The second technique embeds data by writing incremental updates for objects that do not exist in the original data, so that the update has no effect. The data is embedded in the value of the stream objects used in the update.

The third technique embeds data by writing incremental updates with a given length for several objects; then the data can be retrieved by reading the cross-reference section of the update, for it includes the start address of each updated object.

1.2.2 Hiding data in operator values

The above techniques allow to perfectly hide data *if the PDF is opened in a regular PDF viewer*. Sadly, there are tools that allow to decompress PDF data and read it *in clear text*, and most of those techniques then become useless. The following algorithm offers a solution to tackle this issue. Instead of hidden invisible PDF components, it uses values that are already present inside the PDF document.

Justified text and TJ operators S. Zhong *et al.* present a way to create and exploit a secret channel in [3], making use of *justified* text.

They stated that justifying a text (so that it is aligned both with the left and right margin) using a PDF writer would produce random values for the TJ operators that are used to position the characters. It would then be possible to hide data in the least significant bits of some of these TJ operator values. However this works only when the TJ operator values are random and do not contain any pattern.

1.3 Main contributions of this paper

This paper builds on the work by S. Zhong *et al.*, which is presented in [3], that uses the TJ operator values in text stream objects to hide data in PDF

files. The algorithm described in that paper is thoroughly examined for weaknesses. The PDFStego program that is described in the referenced paper is apparently not publicly available or very well hidden in the corners of the internet. An implementation based on this algorithm is therefore developed to test its effectiveness. Besides the demonstration of the weaknesses of the original TJ method, different improvements to the capacity and security are evaluated and implemented. In the end, two new algorithms based on the TJ method are proposed. The first one has a lower capacity but offers better security. The second one offers more capacity while the same level of security is maintained.

1.4 Outline

The next Section 2 gives a general introduction to PDF files and the useful operators that may be relevant for our research. The description of the original TJ algorithm and our implementation of it are described in Section 3. Section 4 focus on the analysis of the original algorithm and Section 5 gives details about our proposed solutions to improve the capacity and security of the algorithm. The conclusions that can be drawn based on the results of our research are given in Section 6. Finally in Section 7 some suggestions for further research in this topic are given.

2 Portable Document Format

The Portable Document Format is a platform independent file format to represent documents. Text and images inside PDF files are displayed in the same way on every platform.

Initially, PDF was a proprietary document format from Adobe and first released in 1993. By July 1, 2008, the International Organization for Standardization (ISO) published PDF as an open standard under number ISO 32000-1:2008. The standard is available from Adobes website [4].

A PDF document consists of a collection of objects that determines the output and functionality of the document. One of the most used objects is the stream object. Text for example is contained in a stream object. Some other objects are numbers, strings, arrays and dictionaries.

2.1 Compression

PDF files are usually compressed in order to save disk space. To be able to view the full source code of the PDF file, one has to decompress the file first. This can be done with programs like pdftk [5] or QPDF [6].

Decompressing a PDF file is an operation that doesn't take much processing time. The decompression of a file with a size of less than 1MB takes only some seconds and even a 1GB file will be decompressed within one minute.

This means that compressing the PDF file does not add extra security when one wants to hide a message or data inside a PDF file.

2.2 Operators

A PDF file contains different operators that can be used to show text as well as position text inside the PDF document. The Tc operator and the Tw operator define the character and word spacing. The Tj operator is used to display (or paint) a text string. The more advanced TJ operator is also used to display a text string, but unlike the simple Tj operator it can control the positioning of individual characters within a text string.

$T_c = 0$ (default)	Character
$T_c = 0.25$	C h a r a c t e r

Figure 1: Tc operator example

2.2.1 Tc operator

This operator is used to control the space between characters and operates on a whole text block. The functionality provided by the Tc operator is used to change the overall density of the text. Within the field of typography, this concept is known as tracking.

The initial value of the operator is set to 0. By changing the value into a positive integer, the space between the characters is increased as can be seen in Figure 1 were the value is set to 0.25. A negative value will decrease the space.

Tc values are expressed in unscaled text space units. The default text space unit is one point (1 pt). Unscaled means it is not dependent on the font size. The Tc value of 0.25 in the example means that the space between each character will be increased by 0.25 pt (with a default text space unit of 1 pt).

2.2.2 Tw operator

The Tw operator is used to set the space between words. It works in the same manner as the Tc operator but only applies to the space character. The default value is 0. An example use of the Tw operator can be found in Figure 2.

Tc values are also expressed in unscaled text space units. The Tw value of 2.5 in the example means that the space between each word is increased by 2.5 pt (with a default text space unit of 1 pt).

$T_w = 0$ (default)	Word Space
$T_w = 2.5$	Word Space

Figure 2: Tw operator example

[(AWAY again)] TJ	AWAY again
[(A) 120 (W) 120 (A) 95 (Y again)] TJ	AWAY again

Figure 3: TJ operator example

2.2.3 TJ operator

The TJ operator is used to display text strings in a PDF file. It contains an array of strings and numbers which respectively consists of the characters and the space values that are used between these characters. The characters are displayed in the same way as when the Tj operator is used. However, for each TJ space value the current text position is altered by subtracting the value from the current position. A negative value means that the next character is moved a bit more to the right which increases the space. A positive value means the next character is moved closer to the previous one which decreases the space. Variable space between characters is often used to create a better looking output. Within the field of typography, this concept is known as kerning. The TJ operator is also used a lot to define the variable space between characters in justified texts.

The TJ space values are expressed in scaled text space units. The default unit is 1/1000 of an em. An em is a unit relative to the specified font size. For example, 1 em with a font size of 12 pt is equal to 12 pt.

An example of the working of the TJ operator can be seen in Figure 3.

2.2.4 Comparison of operators

To find out the properties of some of the operators and the reason why TJ operator values are chosen to hide data into, several PDF files were examined. The presence and frequency of the three discussed operators are shown in Table 1.

Table 1: Appearance of the Tc, Tw and TJ operators in different PDF files

File Operator	1	2	3	4	5	6	7	8
Tc	1272	0	554	2016	87	561	389	976
Tw	963	0	526	1853	0	430	0	765
TJ	668	1171	442	1246	784	598	1036	790

The TJ operator is, in comparison to the Tc and Tw operator, used in every PDF file. Each line of text is represented by one TJ operator. Each TJ operator contains one or more space values. If a text is justified, which means that it is both aligned with the left and right margin, the TJ operator is used more often to introduce variable spacing between words and characters to meet the justification rules. In contrast to this, Tc and Tw values only contain one space value for a block of text. Although Tc and Tw operators can probably be used to hide data in PDF documents, TJ values seem to be the most promising.

3 Implementation of the original method

As a basis for our work we implemented the original TJ algorithm that is described in [3]. The implementation is made available through Github [7].

To give a short overview, the original method uses TJ values between $[-16,16]$ in PDF files created with Jaws PDF to hide data into. Input data is embedded in chunks of 4 bits which corresponds to the values in the range $[1,16]$ after the addition with 1. Only the absolute value is taken into account, the minus sign is ignored. The TJ values between $[-16,16]$ that are used to hide data into are randomly chosen with the use of a Logistic Chaotic Map which act as a Pseudorandom Number Generator (PRNG). All other TJ values between $[-16,16]$ are replaced by values in the same range that are derived from another Logistic Chaotic Map.

3.1 Technical considerations

3.1.1 Python 2

We used Python 2 [8] to create our version of the TJ algorithm, mostly because it offers a convenient syntax and because it usually requires less lines than other scripting languages. Besides, the `re` module provides a nice and practical way to deal with regular expressions (as described below).

In order to perform some specific operations on strings and numbers, we wrote a dedicated class containing several useful methods to split sequences, transcode strings between ASCII codes and numerical forms (binary, decimal, hexadecimal). All those functions are aware of a special parameter: the bit depth (defaults to 4) used to embed numerals as TJ values. The class makes use of the `select` module. It also allows to compute the Secure Hash Algorithm 1 (SHA-1) digest of some strings, as needed by the original method, this is done by the `hashlib` module.

We also wrote a class implementing chaotic maps (used as a PRNG), and allowing the use of a string to work as a seed for the chaotic map.

3.1.2 Parsing the TJ operators

We used the `re` module to parse the TJ operators. First, we parse the TJ blocks using `r'\[(.*)\][ ]?TJ'`.

Then, we parse the block to extract every TJ value from it: `r'[>)](-?[0-9]+)[<()'` (we allow two different conventions: parentheses `(...)` and angular brackets `<...>`, because some PDF creators use one, and some use the other). In order to obtain statistics about the distribution of TJ values, we parse the file several times for each embedding process.

3.1.3 QPDF

QPDF is "a content-preserving PDF transformation system". It allows compressing and uncompressing of PDF stream objects. But its main interest is the QDF format: this format provides plain-text editing of PDF files, for it can rebuild the cross-reference table and the file structure afterwards.

We use the QPDF system to uncompress input files and convert them to the QDF format. Once file editing (changing the TJ values) is complete, we use

QPDF to rebuild a compressed PDF file, with a valid cross-reference table. All those calls to the QPDF stack are performed through Python's `os.system(...)` function.

3.1.4 User-friendliness

In order to bring some user-friendliness to our program, we made use of the `sys` and `optparse` modules.

The program will take any input from `stdin`, for instance passed-in with a UNIX pipe; otherwise, it is possible to use the `-m` (or `--message`) option; if neither of those is used, the program will ask for input.

We used `optparse` to add a lot of useful options and flags, in order to make the program more user-friendly. Additionally, that made it easier for us get all the data for our research.

3.2 Detailing the original method

During the implementation of the original method, we ran into some problems. It appeared to us that the authors had remained vague about some details.

3.2.1 Generating a seed for the chaotic maps

The procedure to generate a seed for the chaotic maps, based on a 10-character long string, was not completely clear. We were suppose to get a number from each character, concatenate those numbers, and add "0." on the left, to obtain a decimal number strictly between 0 and 1.

However, the paper was unclear about how we should turn the characters into numbers. We made the choice not to add any leading zero. That should not have any consequence on the rest of the algorithm.

3.2.2 Finding the end of the message

The authors did not specify how the receiver should know where the data ends, although it is mandatory.

The embedding algorithm specifies that, when all data has been embedded but there are still available TJ operators, they should be filled with random values. However, the sender also embeds a digest of the data (without any trailing random values) as a checksum. Upon extracting, the receiver must check the extracted data digest against the extracted checksum; if the extracted data contains any trailing random value, the digest will not match.

Consequently, the receiver must know where the data ends. The authors did not mention how, so we figured it out ourselves: we use the digest of the key, which gets embedded at the end of the data. The digest works as an ending sequence for the receiver, who now knows where the embedded data ends.

4 Evaluating the TJ method

The techniques that were used to find the weaknesses in the original TJ method are described in this section. The experiments were mainly focused on finding patterns in the TJ space values and the differences that are introduced when these values are changed.

4.1 Data set

To be able to work on the statistical properties of PDF files, a proper dataset was needed. To create this dataset, one of the most popular e-books from the Project Gutenberg [9], the "Adventures of Huckleberry Finn by Mark Twain" [10] was used. The text was justified and some problematic characters were removed to be able to parse the documents more easily. The edited text that was used as the basis of each experiment contained 585,812 characters. Unless stated otherwise, all experiments were performed using PDF documents created from this reference text. The PDF documents were created with LibreOffice [11] as editor and Jaws PDF [12] as PDF writer for the original TJ method. For all documents the same font shape and font size was used. The text was used to create both one-column and two-column PDF documents. The main idea is to use a large enough data set to make the statistical results relevant.

Histograms were created to see if there are differences between the distribution of TJ values in one-column and two-column PDF documents and to determine if it is possible to create a larger reference dataset by combining the data from the one- and two-column documents.

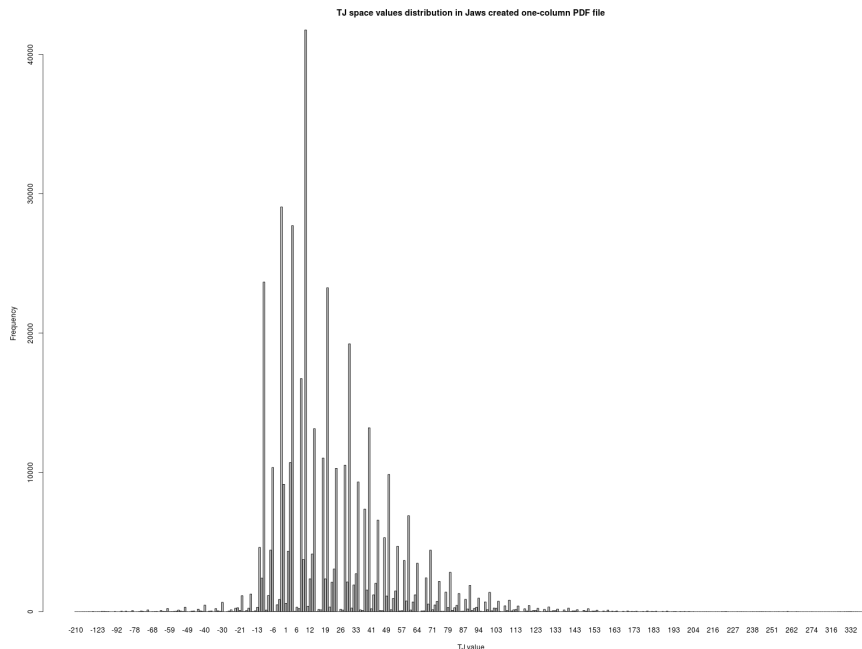


Figure 4: Distribution of TJ space values in an one-column document created with Jaws PDF

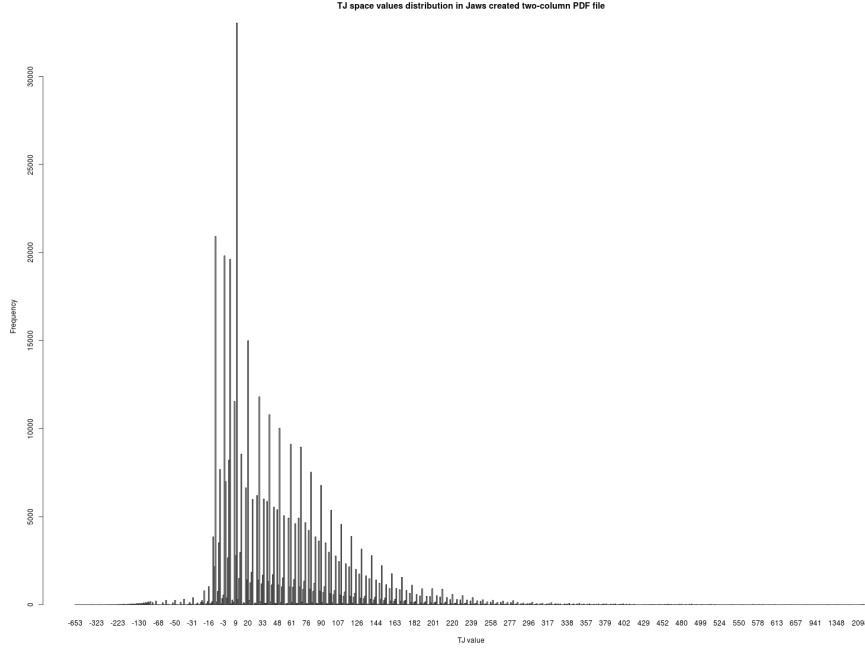


Figure 5: Distribution of TJ space values in a two-column document created with Jaws PDF

Figures 4 and 5 show the distribution of the TJ space values in one-column and two-column documents and Figure 6 shows the distribution of TJ space values from the combined document created with Jaws PDF, the PDF writer that is part of the Original TJ method. As one can see, the distribution is almost the same. All of them follow almost the same pattern and the most frequent values are also the same in the three data sets. So we can use any of these files as a reference for a normal distribution of TJ space values. Therefore we did choose the combined document for general analysis of the normal distribution and the two-column document to compare the difference between a file containing hidden data with a normal file.

4.2 Randomness of TJ values

There was an assumption made in [3] saying that TJ space values between $[-16,16]$ that are used in justified PDF files created by Jaws PDF are random enough to use them as a secret channel to hide data. Based on this assumption, the authors of that paper randomly chose TJ space values between $[-16,16]$ to hide data into and replaced the rest with random numbers within the same range. To verify if a sequence of numbers is random, frequency tests could be used. It is one of the basic ways to check randomness of any sequence by counting the occurrence of each number. If the sequence provides random behaviour, the frequency of each number would be roughly the same. In the list below one can find some statistics about the different TJ space values that are created from the combined document created with Jaws PDF:

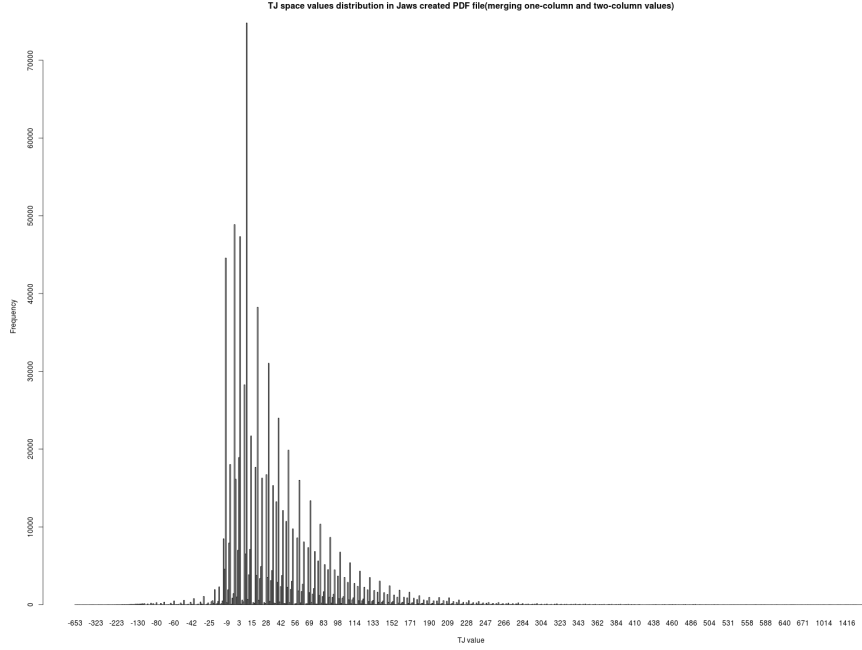


Figure 6: Distribution of TJ space values in the combined document created with Jaws PDF (containing one-column and two-column text)

- 14.6% odd numbers
- 85.4% even numbers
 - 37.5% end with 0
 - 3.9% end with 2
 - 21.6% end with 4
 - 0.6% end with 6
 - 22.1% end with 8

As one can see the odd numbers are not used that often as the even numbers. However there are also some varieties in the even numbers. Ten multipliers are the most frequent even numbers and the numbers ending with six are the least frequent ones which can be considered as outliers (the percentage of their usage is not even 1%).

In Figure 7 one can see that the TJ space values between $[-16,16]$ also follow these percentages. There are numbers which are used very frequently and numbers which are used rarely. As it is shown, the frequency of TJ space values does not follow a unified distribution which results in a non-random sequence. Because the results of the experiment proved differently, we cannot confirm the claim that TJ values between $[-16,16]$ contained in a PDF document created with Jaws PDF are random;

Using our implementation based on the original algorithm, we embedded some text in the PDF document and we checked the output file for the distribution of TJ space values. Figure 8 illustrates that TJ space values in a Jaws

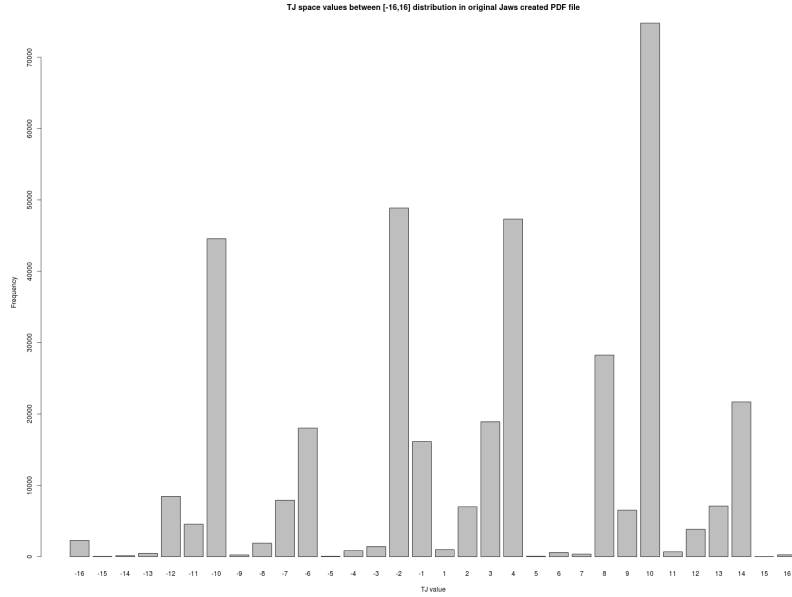


Figure 7: Distribution of TJ space values between $[-16,16]$ in a Jaws PDF file

PDF file containing hidden data behave in a more random way which is different from their original behaviour. This proves that hidden data in a PDF document created with Jaws PDF can be detected by looking at the distribution of TJ values.

4.3 The total line width

There might be other ways, besides looking at the general distribution of TJ values, to detect hidden data in PDF documents. Another possible approach is to look at the line width. Justified text is aligned both with the left and right margin. This could mean that there is a fixed line width which can be calculated.

A line of text contained in a TJ array exists of characters and TJ space values which represent the variable space between those characters. If the total width of all characters in a TJ array is calculated and added to the total sum of all TJ values for that array, one should get a value that represents the total line width. If this value is more or less the same for each line, it should be relatively easy to detect a PDF file which contains hidden data embedded with the TJ method. Even small changes to the line width that wouldn't be visible with the naked eye might be detectable in this way.

Calculating the TJ values should not be a problem. But how can the width of a specific character be determined? One can assume that not every character has the same width. Simple fonts (e.g. Type 1 [13], Type 3 and TrueType [14] fonts) contain a "Widths" key in the font dictionary which defines the character widths or contains a reference to another object that defines the character widths. Figure 9 contains an example. It shows a font dictionary with a "Widths" key that contains a reference to object 6. This object contains the character widths

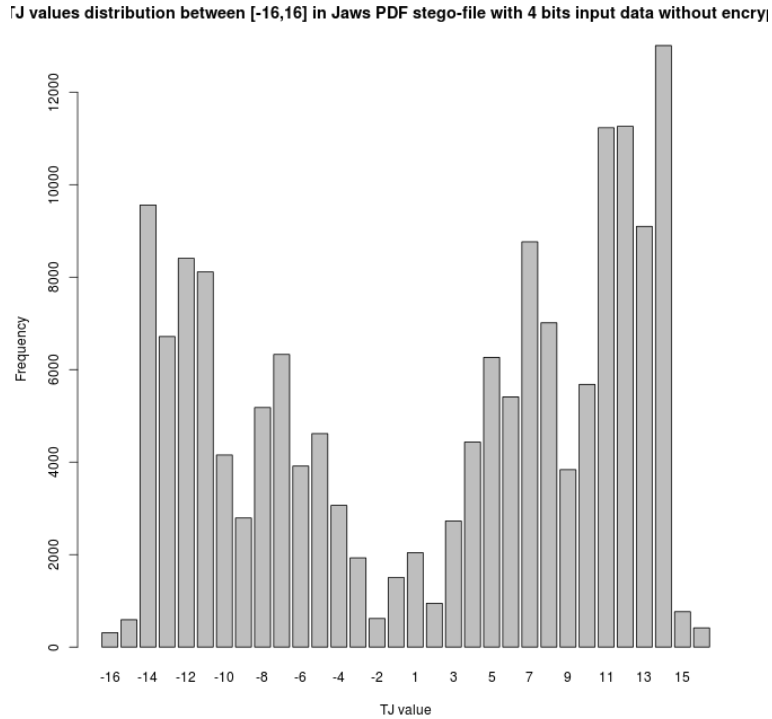


Figure 8: Distribution of TJ space values between [-16,16] in a Jaws PDF file containing hidden data

for the characters of that specific font.

```

2 0 obj
<<
/LastChar 100
/BaseFont /MMIVQW+CMR10
/Subtype /Type1
/Widths 6 0 R
/FontDescriptor 7 0 R
/Type /Font
/FirstChar 45
>>
endobj

6 0 obj [333.3 277.8 500 500 500 500 500 500 500 500 500 500 500 500 500 500 277.8 277.8
277.8 777.8 472.2 472.2 777.8 750 708.3 722.2 763.9 680.6 652.8 784.7 750 361.1
513.9 777.8 625 916.7 750 777.8 680.6 777.8 736.1 555.6 722.2 750 750 1027.8
750 750 611.1 277.8 500 277.8 500 277.8 277.8 500 555.6 444.4 555.6]
endobj

```

Figure 9: Character widths object

A simple experiment was executed to prove the hypothesis that the total line width can be calculated to detect hidden data. A twenty page, two column PDF document was automatically generated with "words" that contain up to nine random characters from the list a, b, c and d. A tool was created to calculate each line width. The width values for the used characters were searched for in

the object that contained the widths and were subsequently hardcoded in the tool. This approach should be adequate enough for this experiment but could be automated at a later time. The last four values in object 6 from Figure 9 are the widths for the characters a, b, c and d in the generated PDF document.

The results of the experiment are shown in Figure 10. The numbers in front are the frequency of the line width values in the PDF. The line width values are the last number in each row. One can distinguish two different ranges of values and two special values. The values between 22099 and 22101 are used for a normal line of text. The values between 21766 and 21768 are used in lines where hyphenation is applied to break a word at the end of the line. The value 4444.2 is the value that is used for the last line. This line does not contain enough characters to justify the text which results in a much lower value. The value 21100.4 is used for the first line which is indented.

It should be clear that most of the lines in a justified text will have an equal width value and that changing the TJ values will affect these line widths. A high count of line widths that don't meet the pattern of the file overall, could be a sign that the PDF document contains hidden data. Due to time constraints, there was no further attempt taken to actually use this information in a more practical way.

```

264 Total line value: 22099.8
229 Total line value: 22100.2
228 Total line value: 22100.0
208 Total line value: 22100.4
154 Total line value: 21766.8
152 Total line value: 21766.4
150 Total line value: 21766.6
149 Total line value: 21767.2
148 Total line value: 21767.0
124 Total line value: 22099.6
101 Total line value: 22100.6
1 Total line value: 4444.2
1 Total line value: 21100.4

```

Figure 10: Line width frequency

4.4 Usefulness of the Logistic Chaotic Maps

One of the prominent parts in the original TJ algorithm is the use of Logistic Chaotic Maps as a source of random numbers. One is used to select a random place to embed data into and another one is used to create random numbers between [1,16] that can be inserted to create redundancy and fill in left over values. It can be called in question if these Logistic Chaotic Maps really add something useful to the steganographic security of the method. It may be the case that it will be more difficult to extract the embedded data when that data is hidden in random places, but Section 4.2 and 4.3 of this report already proved that it does not make it harder to detect the existence of this data when statistical analysis is used.

One might also ask why random values between [1,16] that are created from a Logistic Chaotic Map are used to replace the original values from which the researchers claim that they are already random. It can be argued that useful capacity is lost in return for a form of encryption that is weaker than for example Advanced Encryption Standard (AES). Assuming the results of the executed

experiments are correct, the hidden data is probably even easier to detect because the non-random TJ values are replaced by random values generated from a Logistic Chaotic Map. This means that the steganographic security might be better off without the use of the Logistic Chaotic Map to replace TJ values.

5 Patching and improving the TJ method

5.1 Comparison of different PDF writers

As discussed in Section 4.2, the TJ values inside a PDF file created with Jaws PDF do not show a random behaviour. By analysing the TJ values created by different other PDF writers one can examine if the TJ values created by them can be used to make the method more secure.

PDFCreator

PDFCreator [15] is a PDF writer application for Windows operating systems. It creates a virtual printer, which can be used to print a document to a PDF file. By using PDFCreator to create PDF files we noticed that only 0.3% of the TJ space values that are used in the PDF file were integers and the rest of them were floating point numbers with 5 or 6 numbers behind the point.

At first sight it could be noticed that the numbers after the floating point are the best place to hide data because no matter what the change is, the difference between the new TJ value and the original one would be less than one. But this could be only feasible if the numbers after the floating point provide enough randomness.

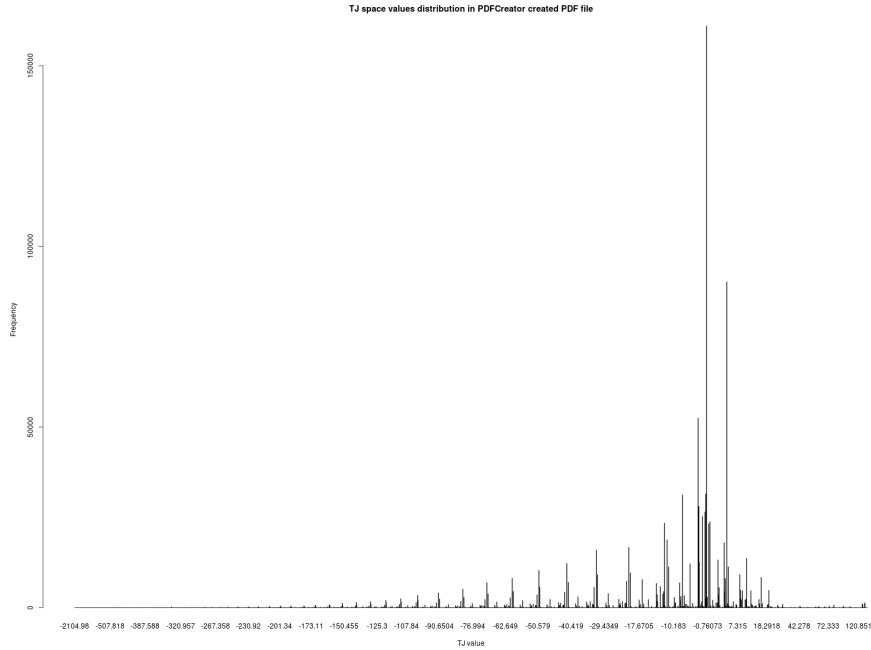


Figure 11: Distribution of TJ space values in a PDFCreator PDF file

Figure 11 illustrates the distribution of TJ space values. As shown, some numbers are grouped together following a special pattern which repeats across the entire data set. Although there are some digits after the floating point, they are used very often (e.g. in our data set, the most frequent value is -0.956417). This means that the changes to the TJ values would be visible in the histogram

when hidden data is embedded.

PDFCreator relies on Ghostscript [16] to generate PDF files. The analysis of TJ values in a PDF document created with CutePDF [17], which is another PDF writer that relies on Ghostscript, gave similar results. It is a reasonable assumption that the same results can be expected from other PDF writers that rely on Ghostscript.

L^AT_EX

L^AT_EX is a document preparation system which is widely used in the academic world. L^AT_EX files are saved as a T_EX file, which can be transformed into a PDF file. PDF_{T_EX} [18], which is part of T_EXLive [19], was used for generating the PDF document from the T_EX file.

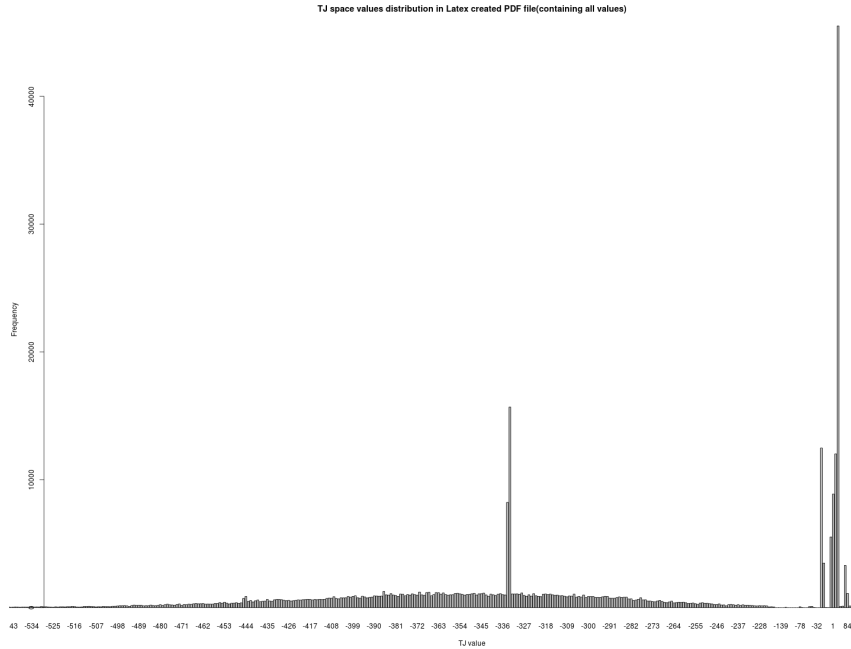


Figure 12: Distribution of TJ space values in a L^AT_EX PDF file

Unlike PDFCreator, L^AT_EX uses integer numbers as TJ values. Figure 12 shows the distribution of TJ space values from the L^AT_EX PDF file. There are a few values causing spikes in the histogram. However, most of the values follow a more random behaviour but with a much lower frequency. There are also a lot of TJ values only used once or twice, which means L^AT_EX uses a wider range of numbers.

In contrast to other PDF writers, the gaps between the TJ values that are used in the PDF file created with L^AT_EX are smaller and less frequent. Using the region of TJ values with a unified distribution, excluding the most frequent values, would make PDF files created with L^AT_EX a promising foundation to build a secure steganographic algorithm based on the TJ method.

5.2 Data encryption

The main goal in (PDF) steganography is eliminating any influence of the input data on the cover-text. Suppose the input data contains, after the binary-decimal conversion, a large frequency of the digit 7 and the cover-text is a Jaws PDF file in which 7 is one of the least frequent values. By embedding the input data in the cover-text, the frequency of the digit 7 in the stego-file would change and be visible in the stego-file's histogram.

When the distribution of TJ values in a PDF document contains one or more patterns, this pattern will change when data is embedded in that document which makes it possible to detect the presence of the hidden data. This is also valid when non-random data is embedded in a PDF document that contains random TJ values. This means that both the original TJ values and the input data should be random to avoid detection by statistical analysis.

The encryption of the input data provides us with a sequence of random data. To prove the effect of using encrypted input data, two stego-files were created. The hidden data of one of them consists of 20KB of cleartext. The hidden data in the other stego-file was encrypted with AES-256-CBC before it was embedded. The hidden data was embedded in chunks of 4 bits. The cover-files were generated from the same $\text{\LaTeX}$ source file. Because of the conclusions of Section 5.1, only the region of TJ values with a unified distribution, excluding the most frequent values, was used to hide data.

Figures 13 and 14 show the distribution of the TJ values in a stego-file containing cleartext input data and encrypted input data. As expected the latter is more close to the original cover-text and keeps its properties.

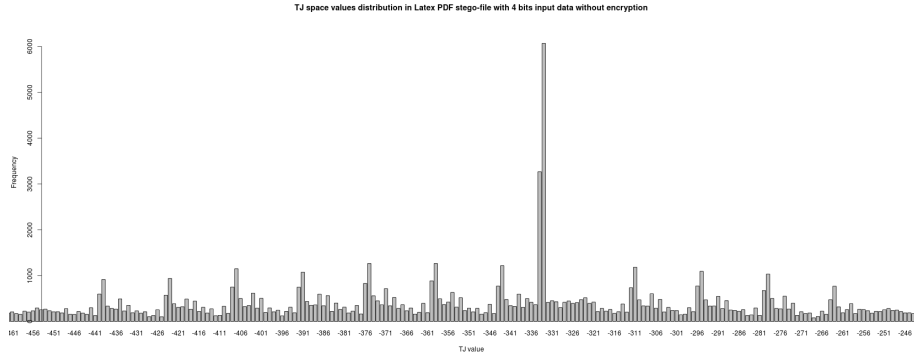


Figure 13: Distribution of TJ values in a $\text{\LaTeX}$ PDF stego-file with 4 bits input data without encryption

5.3 Number of used bits in TJ values

The original algorithm splits the input data into 4 bits, which means that the input data values will vary from 1 to 16 after the conversion to decimal and the addition with 1, as described in [3]. The more bits that are used for each TJ value, the more information can be stored. On the other hand, the more bits that are used for each TJ value, the more distortion will be created in each line of text. This can be visible in the PDF output and the histograms when the

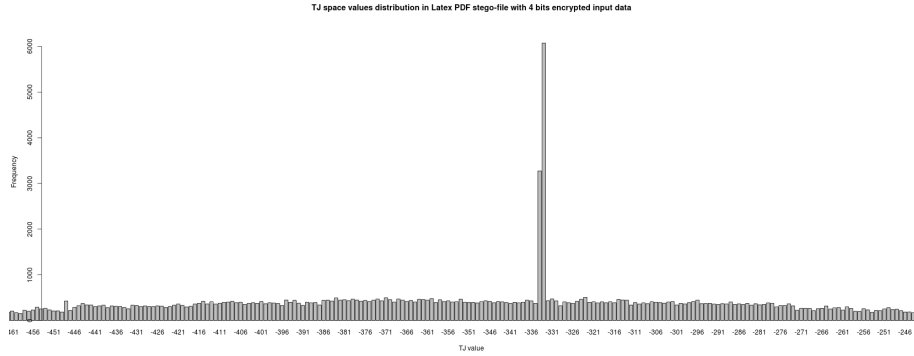


Figure 14: Distribution of TJ values in a $\text{\LaTeX}$ PDF stego_file with 4 bits encrypted input data

distortion reaches a certain boundary. This effect in the output of the PDF file will even be greater when neighbouring lines contain a distortion in the opposite direction.

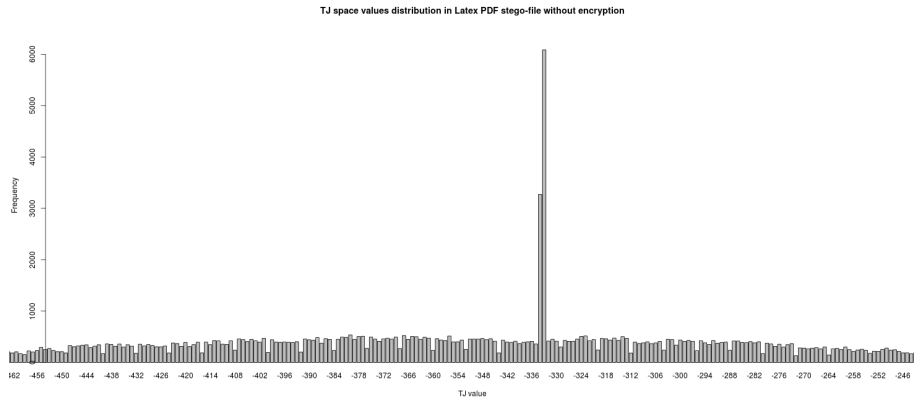


Figure 15: Distribution of TJ values in a $\text{\LaTeX}$ PDF stego_file with 3 bits input data without encryption

Figure 15 illustrates the distribution of TJ values using 3 bit chunks of input data without encryption. If one compares that with figure 13, it can be concluded that 3 bit chunks of input data would be the better choice, although it lowers the available capacity and still contain a distorted histogram.

In the case that input data is encrypted before embedding it in the cover-file, the result changes. Figure 16 and 14 show little difference between the use of 3 or 4 bits of input data when it is encrypted. This experiment shows that it is safe to use chunks of 4 bits of input data when this data is encrypted. Figure 17 proves that the output of a stego-file with input data of 4 bit chunks still looks perfectly aligned.

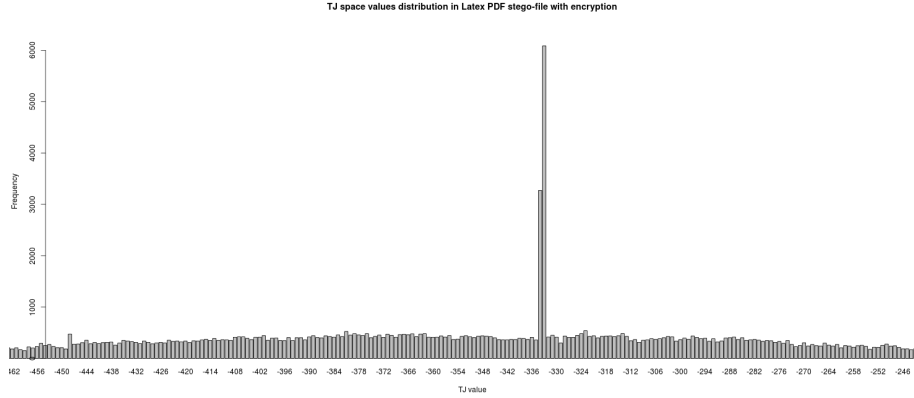


Figure 16: Distribution of TJ values in a $\text{\LaTeX}$ PDF stego_file with 3 bits encrypted input data

Hines let out a whoop, like everybody else, and dropped my wrist and give a big surge to bust his way in and get a look, and the way I lit out and shinned for the road in the dark there ain't nobody can tell.	the river, and it DID seem so good to be free again and all by ourselves on the big river, and nobody to bother us. I had to skip around a bit, and jump up and crack my heels a few times—I couldn't help it; but about the third crack I noticed a sound that
--	---

Figure 17: The output of a stego_file with 4 bits input data and with encryption

5.4 Using most of the TJ values

In the original TJ method only a portion of TJ space values is used for embedding data. Only the TJ values between $[-16, 16]$ were chosen and a certain percentage of them, depending on the value of the redundancy parameter, will not be used to hide data. Figure 18 shows the percentage of TJ values between $[-16, 16]$ in a Jaws PDF file. As it illustrates, more than half of the values are left unused and this even does not include the values that are left out because of the redundancy parameter.

One obvious improvement to create more capacity could be the use of all the TJ values, instead of only the ones between $[-16, 16]$. This can be accomplished by converting the original TJ value to binary, changing the last 4 bits according to the input data and changing the value back to decimal. However, using every TJ value can reveal the presence of hidden data because the normal distribution of TJ values contains some values that are rarely used and some other values that are used very frequently.

For example in the TJ values distribution extracted from a $\text{\LaTeX}$ PDF file (Figure 12), there are few values where the frequency is higher than the others. Most of the other TJ values follow more or less an unified distribution. However, outside the block of evenly distributed values there are values used very rarely or not at all. This can be solved by selecting a region of values that are more or less evenly distributed and skipping the values that create peaks and valleys.

The TJ space values, extracted from a $\text{\LaTeX}$ PDF file (Figure 12), in the range of $[-450, -250]$ follow a more or less unified distribution. By adapting this range to the number of bits used (e.g. $[-447, -257]$ for 4 bits) the crossing of the

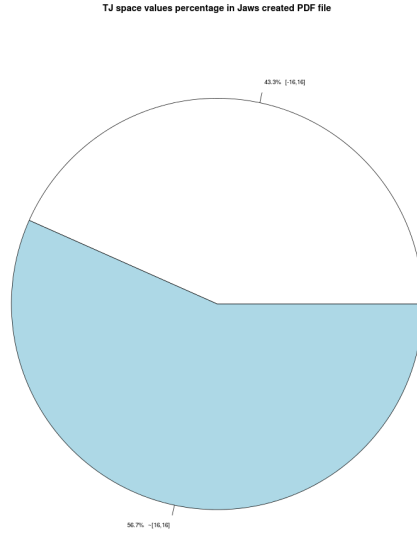


Figure 18: Percentage of TJ space values in a Jaws PDF file

established boundaries can be prevented. Finally, by using the ranges $[-447, -337]$ and $[-320, -257]$, the values -334 and -333 , which are highly frequent values, can be avoided.

Because the distribution of TJ values in a Jaws PDF document (Figure 6) follows a pattern of high peaks and deep valleys, the same technique as applied to PDF documents created with $\text{\LaTeX}$ cannot be implemented successfully. Although the use of all TJ values in a Jaws PDF document would change the distribution even more, it wouldn't matter that much because it was already proved in Section 4 that hidden data could be detected with the use of statistical analysis. Therefore it can be assumed that it should be easy to increase the available capacity while keeping the same level of security, taking into consideration that the steganographic security is not that high.

5.5 Compensating the line width by changing TJ values

As discussed in Section 4, the line width in a PDF file with justified text would be more or less the same and wouldn't contain a wide range of values.

When the TJ values are replaced while hiding the message inside the PDF file, the probability that the values are different and that the total line width is changed is very high. That means that the text is not perfectly justified any more. However, it may not be visible for humans by looking at it. The left alignment would be satisfied because the first character has an absolute position. The right alignment however, would vary for lines with changed TJ values because the characters after the first one are placed relatively to the previous character based on the TJ value.

The solution for this problem would be to withhold some TJ values to compensate for the line width. The total of all changed TJ values for one line can be compared to the total of the original TJ values for that line. The difference in

width can be compensated for by distributing this difference over the reserved TJ values. In a worst case scenario where one TJ value is used to compensate for the change introduced by another TJ value, 50% of the capacity will be lost. However, smarter ways can be invented to the point that only one TJ value is needed to compensate for the total difference in line width.

5.6 Random start and input positions

Imagine the case where the size of hidden data is considerably small and is hidden in a random place within the stego-file. In this situation, finding the start position to analyse afterwards would be more difficult. Although it does not change the distribution of the TJ values and does not add anything to the steganographic security, it can make it harder to extract the hidden data. The placement of input data and line width compensation values within each line can also be randomized. For this randomization functionality of start and input positions, the same or a different password can be used as for the encryption part. By implementing this functionality in a specific way, one can make it also much harder and cumbersome for an attacker to execute a brute force attack. These ideas are not implemented or tested yet, but they may be a better alternative for the randomization features that are introduced by the Logistic Chaotic Maps that are used in the original implementation because no redundancy is introduced and thus no capacity is lost.

5.7 The new algorithm

Sections 5.1 - 5.6 have introduced improvements to the steganographic algorithm described in [3]. Although the research question focuses more on capacity than security, a lot of the described improvements are in the field of steganographic security. The reason for this is that the original TJ algorithm seems to be relatively weak. It might be hard to notice hidden data by looking at the PDF output or uncompressed source code, it is clearly visible when doing statistical analysis on the file.

The improved and recommended algorithm to hide data in PDF documents is a combination of the original TJ algorithm and the improvements described in Sections 5.1 - 5.6. It uses PDF documents created from $\text{\LaTeX}$ source files as a basis and uses chunks of 4 bits to hide the input data in TJ values. The input data is encrypted before it is embedded in the stego-file to keep the distribution of TJ values as close as possible to the original distribution. Two ranges of TJ values $[-447, -337]$ and $[-320, -257]$ were selected as possible sources to hide the input data. This is done to avoid changing TJ values that have a very low or very high frequency. This also means that most TJ values will be used to hide data instead of only the values between $[-16, 16]$. To make it impossible to notice the difference in the PDF output and to counter an attack that calculates and compares the line widths, some TJ values will be used to compensate for the changes in the line widths that are introduced. At last, the randomization and redundancy features that are part of the original algorithm are discarded in favour of extra capacity. Alternative randomization features described in Section 5.6 can be used instead.

5.8 Evaluating the new algorithm

Multiple improvements to the steganographic security have been incorporated in the new algorithm to protect it against statistical analysis but this does not mean that it is secure against other methods that are not yet researched during the project. One method described here could be to look at the TJ value distribution of specific character pairs.

Although several improvements to the embedding capacity have been incorporated in the new algorithm, it is not yet proven how much capacity gain has been obtained. This will also be described in this section.

5.8.1 Randomness of TJ values for character pairs

A text is a structured collection of characters that form words, sentences, paragraphs and so on. One does not really expect randomness within a text. Important concepts within typography are kerning and tracking. As explained before in Section 2, kerning is the process of adjusting the spacing between character pairs to generate a better looking output and tracking is the process of adjusting the spacing in a group of characters to change the overall density.

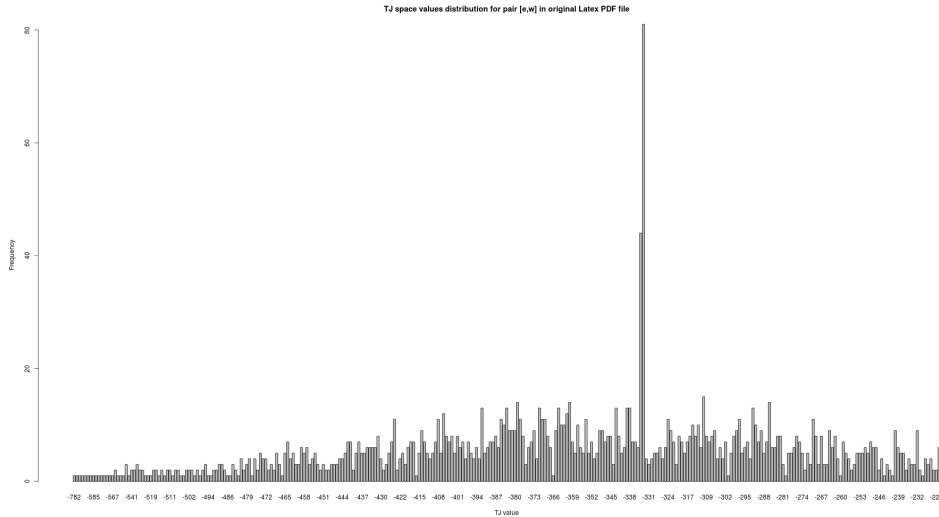


Figure 19: Distribution of TJ values for the e-w pair in a $\text{\LaTeX}$ PDF file without hidden data

These concepts might give some expectation that certain character pairs prefer specific TJ values more than others. In that case, one might expect to find patterns within TJ values for certain character pairs, which can be used to detect hidden data. To test this hypothesis, a tool was developed to extract all TJ values for each character pair in a PDF file. Histogram charts were created to check the distribution of TJ values for certain character pairs. This has been done for the five character pairs in a $\text{\LaTeX}$ PDF document that contained the most unique TJ values (e.g. e-t, e-w, t-t, n-t, and d-t). The results of the e-w and d-t pairs are displayed in Figures 19 to 22. It is hard to make a statement about these histograms. Although one can see some differences between the

histograms that show the distribution of TJ values for the PDF files with and without hidden data, there are no real patterns visible. More research is needed to be able to determine if the distribution of TJ values for specific character pairs can be used to detect hidden data.

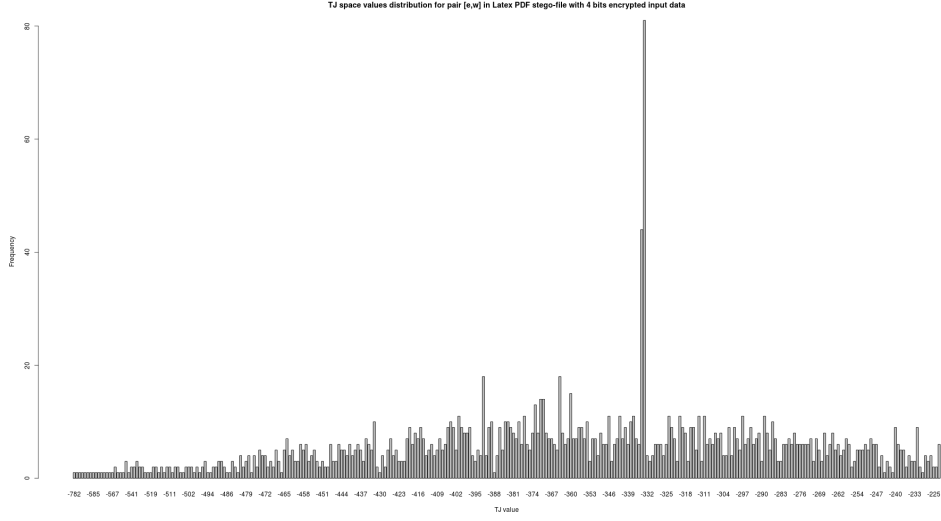


Figure 20: Distribution of TJ values for the e-w pair in a $\text{\LaTeX}$ PDF file with hidden data

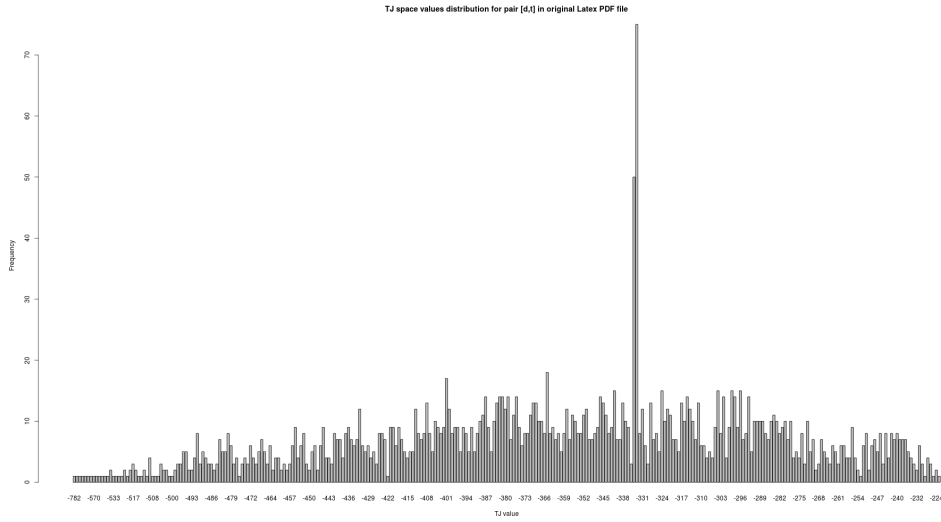


Figure 21: Distribution of TJ values for the d-t pair in a $\text{\LaTeX}$ PDF file without hidden data

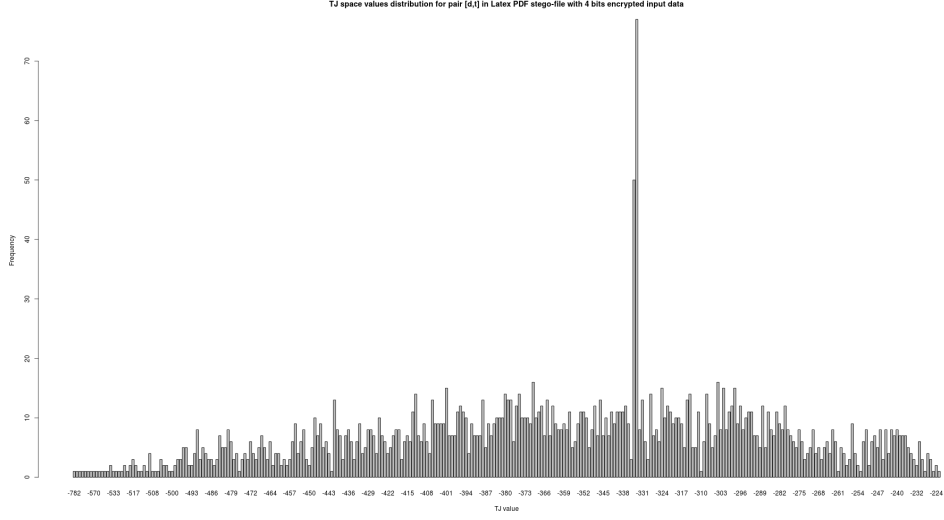


Figure 22: Distribution of TJ values for the d-t pair in a L^AT_EX PDF file with hidden data

5.8.2 Comparison of the available capacity

The calculation of the embedding capacity of the original algorithm is displayed in Equation 1. The amount of characters in a PDF document is denoted by cm . The percentage of kerning pairs, character pairs that contain a TJ value, is denoted by $sk\%$ and $se\%$ can be seen as the percentage of useful TJ values (i.e. TJ values in the range $[-16,16]$). The parameter of redundancy is contained in $pr\%$.

$$Capacity = ((cm - cm \times sk\%) \times se\%) \times (1 - pr\%) \quad (1)$$

Equation 2 can be used to calculate the embedding capacity of the improved algorithm without the width compensation. The useful range of TJ values is denoted by $ra\%$. Equation 3 changed Equation 2 by incorporating the width compensation, which is denoted by $wc\%$.

$$Capacity = ((cm - cm \times sk\%) \times ra\%) \quad (2)$$

$$Capacity = ((cm - cm \times sk\%) \times ra\%) \times (1 - wc\%) \quad (3)$$

Two stego-files were created for a more practical example of calculating the embedding capacity. The first stego-file was created with Jaws PDF and was used to test the embedding capacity of the original TJ algorithm. The second stego-file was created from a L^AT_EX document and was used to test the embedding capacity of the improved algorithm, excluding the line width compensation. Both PDF documents contained the same text as described in Section 4.1. As both methods use data chunks of 4 bits, the capacity can be easily compared by counting and comparing the useful TJ values.

The Jaws PDF document has 442,401 TJ values from which 106,706 can be used to embed data, which means it can embed $106,706 \times 4 \div 8 = 53,353$

bytes. The PDF file created from the L^AT_EX source document has 147,458 TJ values from which 59,110 can be used to embed data, which means it can embed $59,110 \times 4 \div 8 = 29,555$ bytes. This means that the original method wins by a great margin in terms of embedding capacity.

5.8.3 A capacity versus security trade-off

Notwithstanding the capacity improvements in the new algorithm, it turns out that the original algorithm still has a lot more embedding capacity. This is primarily because the Jaws PDF document contains roughly three times the TJ value count of the L^AT_EX PDF file.

The new algorithm is clearly more secure than the original one but has a lower embedding capacity. However, this paper has shown different ways to be able to increase the capacity that also can be applied to the original algorithm. This means that it is still possible to increase the capacity while keeping the same level of security.

When the original algorithm is changed by discarding the randomization and redundancy features that are part of the original algorithm and by using all TJ values, a lot of extra capacity can be gained. Encryption and the alternative randomization features described in Section 5.6 can be used to add some, non-steganographic, security. As the original TJ algorithm has already been broken and does not contain any protection against statistical analysis, these changes will at least keep the same level of security and will add a lot of capacity. The embedding capacity will be $442,401 \times 4 \div 8 = 221,200.5$ bytes. This is roughly four times more than with the original algorithm.

Dependent on what is more important, steganographic security or capacity, one can choose one of the two improved versions of the original TJ method to hide data in PDF files.

6 Conclusions

The first conclusion that can be drawn from the results of our research is that the TJ values between $[-16,16]$ in justified PDF documents created with Jaws PDF are not random in contrast to what the creators of the original TJ method state. This is the main weakness that we exploited to detect hidden data in stego-files created with the original TJ method. The steganographic security of the original TJ method is therefore not very high.

A conclusion that follows the previous one is that the Logistic Chaotic Maps do not provide any real steganographic security. It may be more difficult to reconstruct the embedded data, but the presence of this hidden data was very visible when doing statistical analysis on the distribution of the TJ values.

Another conclusion that can be drawn from the results of our research is that PDF documents created from $\text{\LaTeX}$ source files do produce a more random sequence of TJ values which can be used to hide data without changing the general distribution of TJ values when the input data is also random. This can be accomplished by encrypting the input data before embedding it in the stego-file.

From the results of our research we can also conclude that a PDF document is very structured and that this makes it difficult to hide data into it that cannot easily be detected. An example of this is the line width calculation. Another one is the statistical properties of TJ values within PDF documents created with a specific PDF writer. One has to take care of all these details to create a secure steganographic method based on PDF documents.

A final important but obvious conclusion that can be drawn from the results of our research is that there is a trade-off between steganographic security and capacity. Because not everyone has the same needs, we propose two different improved versions of the TJ method to hide data in PDF documents.

The first method, described in Section 5.7, is more secure and can prevent the detection of hidden data when statistical analysis is performed on the distribution of the TJ values. However, the capacity is lower and there still may be some other ways to detect the hidden data.

The second method offers roughly four times the capacity as the original TJ method while still keeping the same level of security. This capacity has been gained by discarding some limitations and replacing security features that did not work properly by more efficient ones. There is no way to detect hidden data by looking at the output or the source code of the PDF document. However, when doing statistical analysis on the TJ values, the hidden data can be detected easily. This improved version of the original TJ method, which is more clearly explained in Section 5.8.3, can be seen as the answer to the research question of this project:

How can the steganographic embedding capacity in PDF files be increased by altering the existent algorithms while keeping the same level of security?

7 Further research

Due time constraints we where not able to conduct all the experiments that we wanted to conduct. There is still a lot of research that can be done.

Although we did compare a few PDF writers, there are many more that we didn't look at. It could be very well possible that one of them has properties that can be used to create more capacity or a more secure steganographic method.

We also took a quick look at the statistical properties of TJ values from specific character pairs. However, we were not able to make any hard conclusions about our results on that part and more research is needed. We do think that this can be a way to break the security of our improved method. A lot of research can also be done to find other ways to break the security of our improved method.

We did research the possibilities of detecting hidden data in PDF documents that uses the TJ method. However we did not create tools that can automate the detection. Formulas must be created from a baseline of a normal distribution of TJ values to be able to automate this detection.

Finally, it is maybe worth looking at a way to develop a PDF printer that creates normal PDF files that have matching properties with PDF files that contain hidden data. An example of this could be a PDF printer that creates random TJ values. However, the PDF specification is that enormous that it will consume much time.

Ideally one would developed both, a PDF printer and a PDF steganographic application to adjust parameters of both accordingly. The PDF printer could be published and promoted to get a small market share of some percent. The PDF steganographic application could be kept secret to use it for secret messages. However, it is also possible to publish the PDF steganographic application, but then users of the PDF printer could be suspicious of hiding data.

A List of Acronyms

AES Advanced Encryption Standard
ASCII American Standard Code for Information Interchange
ISO International Organization for Standardization
PDF Portable Document Format
PRNG Pseudorandom Number Generator
SHA-1 Secure Hash Algorithm 1

References

- [1] I-Shi Lee and Wen-Hsiang Tsa. A new approach to covert communication via pdf files. *Signal Processing*, 90:557–565, 2010.
- [2] Hongmei Liu, Lei Li, Jian Li, and Jiwu Huang. Three novel algorithms for hiding data in pdf files based on incremental updates. Technical report, Sun Yat-sen University, Guangzhou, China, 2007.
- [3] Shangping Zhong, Xueqi Cheng, and Tierui Chen. Data hiding in a kind of pdf texts for secret communicationl. *International Journal of Network Security*, 4(1):17–26, 2007.
- [4] Pdf reference and adobe extensions to the pdf specification. Website. http://www.adobe.com/devnet/pdf/pdf_reference.html.
- [5] pdftk the pdf toolkit. Website. <http://www.pdflabs.com/tools/pdftk-the-pdf-toolkit/>.
- [6] Qpdf. Website. <http://qpdf.sourceforge.net>.
- [7] Pdf hide. Website. https://github.com/ncanceill/pdf_hide.git.
- [8] Python 2.7.3. Website. <http://www.python.org/getit/releases/2.7.3/>.
- [9] Project gutenber. Website. <http://www.gutenberg.org/>.
- [10] Adventures of huckleberry finn by mark twain. Website. <http://www.gutenberg.org/ebooks/76>.
- [11] Libreoffice 3.6.3.2. Website. <http://www.libreoffice.org/>.
- [12] Jaws pdf creator v5.0. Website. <http://www.jawspdf.com/>.
- [13] Adobe type 1 font format. Website. http://partners.adobe.com/public/developer/en/font/T1_SPEC.PDF.
- [14] Truetype reference manual. Website. <https://developer.apple.com/fonts/TTRefMan/index.html>.
- [15] Pdcreator 1.6.0. Website. <http://www.pdfforge.org/pdfcreator>.
- [16] Ghostscript. Website. <http://www.ghostscript.com/>.

- [17] Cutenpdf writer 3.0. Website. <http://www.cutepdf.com/products/cutepdf/writer.asp>.
- [18] pdftex 3.1415926-1.40.10-2.2. Website. <http://www.tug.org/applications/pdftex/>.
- [19] Tex live 2009. Website. <http://www.tug.org/texlive/>.